```
GUJJA RAJU
2403A52018
CSE AIML B-02
```

```
import pandas as pd

file_path = '/content/UCI_Credit_Card.csv'
df = pd.read_csv(file_path)

print(f"Successfully loaded data from {file_path}. Dataset shape: {df.shape}")
display(df.head())
```

```
Successfully loaded data from /content/UCI_Credit_Card.csv. Dataset shape: (30000, 25)
```

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	...	BILL_AMT4	BILL_AMT5	BILL_AMT6	PAY_AMT1	P
0	1	20000.0	2	2	1	24	2	2	-1	-1	...	0.0	0.0	0.0	0.0	
1	2	120000.0	2	2	2	26	-1	2	0	0	...	3272.0	3455.0	3261.0	0.0	
2	3	90000.0	2	2	2	34	0	0	0	0	...	14331.0	14948.0	15549.0	1518.0	
3	4	50000.0	2	2	1	37	0	0	0	0	...	28314.0	28959.0	29547.0	2000.0	
4	5	50000.0	1	2	1	57	-1	0	-1	0	...	20940.0	19146.0	19131.0	2000.0	

5 rows × 25 columns

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 25 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   ID                          30000 non-null  int64
 1   LIMIT_BAL                   30000 non-null  float64
 2   SEX                         30000 non-null  int64
 3   EDUCATION                   30000 non-null  int64
 4   MARRIAGE                    30000 non-null  int64
 5   AGE                         30000 non-null  int64
 6   PAY_0                       30000 non-null  int64
 7   PAY_2                       30000 non-null  int64
 8   PAY_3                       30000 non-null  int64
 9   PAY_4                       30000 non-null  int64
 10  PAY_5                       30000 non-null  int64
 11  PAY_6                       30000 non-null  int64
 12  BILL_AMT1                   30000 non-null  float64
 13  BILL_AMT2                   30000 non-null  float64
 14  BILL_AMT3                   30000 non-null  float64
 15  BILL_AMT4                   30000 non-null  float64
 16  BILL_AMT5                   30000 non-null  float64
 17  BILL_AMT6                   30000 non-null  float64
 18  PAY_AMT1                    30000 non-null  float64
 19  PAY_AMT2                    30000 non-null  float64
 20  PAY_AMT3                    30000 non-null  float64
 21  PAY_AMT4                    30000 non-null  float64
 22  PAY_AMT5                    30000 non-null  float64
 23  PAY_AMT6                    30000 non-null  float64
 24  default.payment.next.month  30000 non-null  int64
dtypes: float64(13), int64(12)
memory usage: 5.7 MB
None
```

```
display(df.describe())
```

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	
count	30000.000000	30000.000000	30000.000000	30000.000000	30000.000000	30000.000000	30000.000000	30000.000000	30000.0
mean	15000.500000	167484.322667	1.603733	1.853133	1.551867	35.485500	-0.016700	-0.133767	-0.1
std	8660.398374	129747.661567	0.489129	0.790349	0.521970	9.217904	1.123802	1.197186	1.1
min	1.000000	10000.000000	1.000000	0.000000	0.000000	21.000000	-2.000000	-2.000000	-2.0
25%	7500.750000	50000.000000	1.000000	1.000000	1.000000	28.000000	-1.000000	-1.000000	-1.0
50%	15000.500000	140000.000000	2.000000	2.000000	2.000000	34.000000	0.000000	0.000000	0.0
75%	22500.250000	240000.000000	2.000000	2.000000	2.000000	41.000000	0.000000	0.000000	0.0
max	30000.000000	1000000.000000	2.000000	6.000000	3.000000	79.000000	8.000000	8.000000	8.0

8 rows × 25 columns

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Drop the 'ID' column as it's just an identifier and not a feature
df_model = df.drop('ID', axis=1)

# Define features (X) and target (y)
X = df_model.drop('default.payment.next.month', axis=1)
y = df_model['default.payment.next.month']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print(f"Training set shape: {X_train.shape}")
print(f"Testing set shape: {X_test.shape}")
```

```
Training set shape: (24000, 23)
Testing set shape: (6000, 23)
```

```
# Initialize and train a Decision Tree Classifier
dt_classifier = DecisionTreeClassifier(random_state=42)
dt_classifier.fit(X_train, y_train)

# Make predictions on the test set
y_pred = dt_classifier.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

print(f"Decision Tree Classifier Accuracy: {accuracy:.4f}")
print("\nClassification Report:")
print(report)
```

```
Decision Tree Classifier Accuracy: 0.7145

Classification Report:
              precision    recall  f1-score   support

           0       0.83      0.80      0.81      4673
           1       0.37      0.41      0.39      1327

    accuracy                           0.71      6000
   macro avg       0.60      0.61      0.60      6000
weighted avg       0.73      0.71      0.72      6000
```

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Drop the 'ID' column as it's just an identifier and not a feature
df_model = df.drop('ID', axis=1)

# Define features (X) and target (y)
X = df_model.drop('default.payment.next.month', axis=1)
y = df_model['default.payment.next.month']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

```
print(f"Training set shape: {X_train.shape}")
print(f"Testing set shape: {X_test.shape}")
```

```
Training set shape: (24000, 23)
Testing set shape: (6000, 23)
```

```
# Initialize and train a Decision Tree Classifier
dt_classifier = DecisionTreeClassifier(random_state=42)
dt_classifier.fit(X_train, y_train)

# Make predictions on the test set
y_pred = dt_classifier.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

print(f"Decision Tree Classifier Accuracy: {accuracy:.4f}")
print("\nClassification Report:")
print(report)
```

```
Decision Tree Classifier Accuracy: 0.7145

Classification Report:
              precision    recall  f1-score   support

           0       0.83      0.80      0.81      4673
           1       0.37      0.41      0.39      1327

    accuracy                           0.71      6000
   macro avg       0.60      0.61      0.60      6000
weighted avg       0.73      0.71      0.72      6000
```

```
# --- Random Forest Classifier ---
print("\n--- Training Random Forest Classifier ---")
# Initialize and train a Random Forest Classifier
rf_classifier = RandomForestClassifier(random_state=42, n_estimators=100) # Using 100 estimators as a common default
rf_classifier.fit(X_train, y_train)

# Make predictions on the test set
y_pred_rf = rf_classifier.predict(X_test)

# Evaluate the model
accuracy_rf = accuracy_score(y_test, y_pred_rf)
report_rf = classification_report(y_test, y_pred_rf)

print(f"Random Forest Classifier Accuracy: {accuracy_rf:.4f}")
print("\nClassification Report (Random Forest):")
print(report_rf)
```

```
--- Training Random Forest Classifier ---
Random Forest Classifier Accuracy: 0.8120

Classification Report (Random Forest):
              precision    recall  f1-score   support

           0       0.84      0.94      0.89      4673
           1       0.63      0.36      0.46      1327

    accuracy                           0.81      6000
   macro avg       0.74      0.65      0.67      6000
weighted avg       0.79      0.81      0.79      6000
```

```
from sklearn.ensemble import AdaBoostClassifier, RandomForestClassifier

# --- AdaBoost Classifier ---
print("\n--- Training AdaBoost Classifier ---")
# Initialize and train an AdaBoost Classifier
adaboost_classifier = AdaBoostClassifier(random_state=42)
adaboost_classifier.fit(X_train, y_train)

# Make predictions on the test set
y_pred_ada = adaboost_classifier.predict(X_test)

# Evaluate the model
accuracy_ada = accuracy_score(y_test, y_pred_ada)
report_ada = classification_report(y_test, y_pred_ada)

print(f"AdaBoost Classifier Accuracy: {accuracy_ada:.4f}")
```

```
print("\nClassification Report (AdaBoost):")
print(report_ada)
```

```
--- Training AdaBoost Classifier ---
AdaBoost Classifier Accuracy: 0.8167

Classification Report (AdaBoost):
              precision    recall  f1-score   support

           0       0.83      0.95      0.89      4673
           1       0.67      0.33      0.45      1327

    accuracy                           0.82      6000
   macro avg       0.75      0.64      0.67      6000
weighted avg       0.80      0.82      0.79      6000
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

# Define the parameter grid for Random Forest
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20] # None means nodes are expanded until all leaves are pure or until all leaves contain less
}

# Initialize a RandomForestClassifier
rf_base = RandomForestClassifier(random_state=42)

# Initialize GridSearchCV with 5-fold cross-validation
grid_search = GridSearchCV(estimator=rf_base, param_grid=param_grid, cv=5, scoring='accuracy', n_jobs=-1, verbose=2)

print("\n--- Performing GridSearchCV for Random Forest ---")
# Fit GridSearchCV to the training data
grid_search.fit(X_train, y_train)

# Print the best parameters and best score
print(f"Best Parameters found: {grid_search.best_params_}")
print(f"Best Cross-validation Accuracy: {grid_search.best_score_:.4f}")

# Get the best model
best_rf_model = grid_search.best_estimator_

# Make predictions on the test set with the best model
y_pred_best_rf = best_rf_model.predict(X_test)

# Evaluate the best model on the test set
accuracy_best_rf = accuracy_score(y_test, y_pred_best_rf)
report_best_rf = classification_report(y_test, y_pred_best_rf)

print(f"\nBest Random Forest Classifier (Tuned) Accuracy on Test Set: {accuracy_best_rf:.4f}")
print("\nClassification Report (Tuned Random Forest):")
print(report_best_rf)
```

```
--- Performing GridSearchCV for Random Forest ---
Fitting 5 folds for each of 9 candidates, totalling 45 fits
Best Parameters found: {'max_depth': 10, 'n_estimators': 200}
Best Cross-validation Accuracy: 0.8204

Best Random Forest Classifier (Tuned) Accuracy on Test Set: 0.8172

Classification Report (Tuned Random Forest):
              precision    recall  f1-score   support

           0       0.84      0.95      0.89      4673
           1       0.67      0.35      0.46      1327

    accuracy                           0.82      6000
   macro avg       0.75      0.65      0.67      6000
weighted avg       0.80      0.82      0.79      6000
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Prepare data for plotting
plot_df = comparison_df.set_index('Model').transpose()

# Create a figure and a set of subplots
fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(20, 12), sharey=False)
axes = axes.flatten()
```

```
metrics = ['Accuracy', 'Precision (Class 1)', 'Recall (Class 1)', 'F1-Score (Class 1)', 'ROC-AUC']

for i, metric in enumerate(metrics):
    if metric in plot_df.index:
        sns.barplot(x=plot_df.columns, y=plot_df.loc[metric], ax=axes[i], palette='viridis', legend=False)
        axes[i].set_title(f'{metric} Comparison', fontsize=14)
        axes[i].set_ylabel(metric, fontsize=12)
        axes[i].set_xlabel('Model', fontsize=12)
        axes[i].tick_params(axis='x', rotation=45)
        for container in axes[i].containers:
            axes[i].bar_label(container, fmt='%.4f')

# Hide any unused subplots
for j in range(len(metrics), len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.suptitle('Model Performance Comparison Across Key Metrics', y=1.02, fontsize=18)
plt.show()
```

```
/tmp/ipykernel_252/189707565.py:15: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and
```

```
from sklearn.metrics import roc_auc_score
import pandas as pd

# --- Calculate ROC-AUC for each model ---

# Decision Tree Classifier
y_pred_proba_dt = dt_classifier.predict_proba(X_test)[:, 1]
roc_auc_dt = roc_auc_score(y_test, y_pred_proba_dt)

# AdaBoost Classifier
y_pred_proba_ada = adaboost_classifier.predict_proba(X_test)[:, 1]
roc_auc_ada = roc_auc_score(y_test, y_pred_proba_ada)

# Initial Random Forest Classifier
y_pred_proba_rf = rf_classifier.predict_proba(X_test)[:, 1]
roc_auc_rf = roc_auc_score(y_test, y_pred_proba_rf)

# Tuned Random Forest Classifier
y_pred_proba_best_rf = best_rf_model.predict_proba(X_test)[:, 1]
roc_auc_best_rf = roc_auc_score(y_test, y_pred_proba_best_rf)

# --- Consolidate results into a DataFrame ---

# Extract metrics from classification reports (focus on Class 1 for precision, recall, f1)
report_dt_dict = classification_report(y_test, dt_classifier.predict(X_test), output_dict=True)
report_ada_dict = classification_report(y_test, adaboost_classifier.predict(X_test), output_dict=True)
report_rf_dict = classification_report(y_test, rf_classifier.predict(X_test), output_dict=True)
report_best_rf_dict = classification_report(y_test, best_rf_model.predict(X_test), output_dict=True)

comparison_df = pd.DataFrame({
    'Model': ['Decision Tree', 'AdaBoost', 'Random Forest (Initial)', 'Random Forest (Tuned)'],
    'Accuracy': [
        accuracy_score(y_test, dt_classifier.predict(X_test)),
        accuracy_score(y_test, adaboost_classifier.predict(X_test)),
        accuracy_score(y_test, rf_classifier.predict(X_test)),
        accuracy_score(y_test, best_rf_model.predict(X_test))
    ],
    'Precision (Class 1)': [
        report_dt_dict['1']['precision'],
        report_ada_dict['1']['precision'],
        report_rf_dict['1']['precision'],
        report_best_rf_dict['1']['precision']
    ],
    'Recall (Class 1)': [
        report_dt_dict['1']['recall'],
        report_ada_dict['1']['recall'],
        report_rf_dict['1']['recall'],
        report_best_rf_dict['1']['recall']
    ],
    'F1-Score (Class 1)': [
        report_dt_dict['1']['f1-score'],
        report_ada_dict['1']['f1-score'],
        report_rf_dict['1']['f1-score'],
        report_best_rf_dict['1']['f1-score']
    ],
    'ROC-AUC': [roc_auc_dt, roc_auc_ada, roc_auc_rf, roc_auc_best_rf]
})

print("\n--- Model Comparison ---")
display(comparison_df.set_index('Model').round(4))
```

```
--- Model Comparison ---
```

Model	Accuracy	Precision (Class 1)	Recall (Class 1)	F1-Score (Class 1)	ROC-AUC
Decision Tree	0.7145	0.3694	0.4115	0.3893	0.6075
AdaBoost	0.8167	0.6728	0.3331	0.4456	0.7620
Random Forest (Initial)	0.8120	0.6325	0.3580	0.4572	0.7506
Random Forest (Tuned)	0.8172	0.6652	0.3489	0.4577	0.7733